DealHunter

Technical Documentation — 1st delivery

Assignatura: Projecte Web

Professors: Roberto Garcia i David Sarrat

Curs: 2025 / 2026

Repositori: https://github.com/DealHunterOrgan/DealHunter

Membres: Otto Biel Ródanas, Sergi Arbós Nicolau, Marc Verdugo Soria, Bruno Verdugo Soria, Joel Marcet Cruz, Marc Prats

1. Descripció de l'aplicació

DealHunter és una aplicació web desenvolupada amb Django que permet als usuaris trobar les millors ofertes de videojocs entre diverses plataformes i botigues digitals. L'objectiu principal és centralitzar la informació de preus i disponibilitat de jocs en un únic lloc facilitant la presa de decisions de compra.

L'aplicació integra dues APIs externes per oferir una cobertura àmplia:

- **CheapShark API**: especialitzada en ofertes de Steam i altres botigues PC. Proporciona preus actuals i enllaços directes a les ofertes.
- **RAWG API**: base de dades de videojocs que proporciona informació detallada sobre els jocs, captures de pantalla, tràilers i dades generals. Aquesta API permet disposar d'un catàleg molt més complet que el que oferiria CheapShark per si sola.

Les funcionalitats principals de l'aplicació inclouen:

- Navegació i cerca per catàleg de jocs amb filtres per gènere, plataforma i rang de preus. Així com ordenació per preu, puntuació i novetat.
- Visualització de preus actuals registrats per cada joc a cada botiga.
- Accés directe a la botiga corresponent per completar la compra.
- Pàgina de detall de cada joc amb captures de pantalla, tràiler i comentaris d'usuaris.
- Sistema de ressenyes: els usuaris autenticats poden afegir, editar i eliminar comentaris amb valoració.
- Gestió d'una llista de desitjos (Wishlist) personalitzada per a usuaris autenticats.
- Registre, autenticació i gestió del perfil d'usuari incloent selecció d'avatar i canvi de contrasenya.
- Interfície d'administració de Django per gestionar totes les entitats del model.

2. Model de dades

2.1 Entitats principals

Game

Emmagatzema la informació principal de cada videojoc obtinguda de les APIs externes.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic intern
title	CharField	Títol del joc
api_ID	CharField (unique)	Identificador a la CheapShark API
steam_appid	CharField (nullable)	ID d'Steam per consultar la RAWG API

Camp	Tipus	Descripció
score	DecimalField	Puntuació Metacritic (0–100)
launch_date	DateField (nullable)	Data de llançament
cover_url	URLField	URL de la imatge de portada
genres	ManyToManyField → Genre	Gèneres associats
platforms	ManyToManyField → Platform	Plataformes associades

Shop

Representa les botigues digitals (Steam, Epic Games, GOG, etc.) on es poden adquirir els jocs.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
name	CharField	Nom de la botiga
api_ID	CharField (unique)	Identificador a CheapShark
service_state	BooleanField	Indica si la botiga és activa
logo_url	URLField	URL del logotip de la botiga

Availability

Taula intermèdia entre Game i Shop que registra la disponibilitat i els preus (actuals i mínims) d'un joc en una botiga concreta. Aquesta entitat és el nucli del model de dades.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
game (FK)	ForeignKey → Game	Joc al qual fa referència
shop (FK)	ForeignKey → Shop	Botiga on és disponible
current_price	DecimalField	Preu actual de l'oferta
offer_url	URLField	Enllaç directe a l'oferta

Genre

Representa els gèneres de joc (Acció, RPG, Estratègia...), obtinguts de la RAWG API i l'API de Steam.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
name	CharField (unique)	Nom del gènere

Platform

Representa les plataformes de joc (Steam, GOG, Epic Games, etc.), inferides a partir de la botiga d'origen de cada oferta.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
name	CharField	Nom de la plataforma

Wishlist

Taula intermèdia que relaciona usuaris autenticats (User de Django) amb jocs, permetent a cada usuari guardar els jocs que li interessen.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
user (FK)	ForeignKey → User	Usuari propietari de la wishlist
game (FK)	ForeignKey → Game	Joc afegit a la llista
desired_price	DecimalField	Preu objectiu de l'usuari

Review

Permet als usuaris autenticats deixar ressenyes amb text i valoració per a cada joc.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
game (FK)	ForeignKey → Game	Joc referenciat
user (FK)	ForeignKey → User	Autor de la ressenya
content	TextField	Text de la ressenya (màx. 1000 caràcters)
rating	IntegerField	Valoració de l'1 al 5 (per defecte 5)
created_at	DateTimeField	Data de creació (auto)

Profile

Perfil estès de l'usuari, creat automàticament en el registre, que emmagatzema preferències visuals.

Camp	Tipus	Descripció
id (PK)	AutoField	Identificador únic
user	OneToOneField → User	Usuari associat
avatar_url	CharField	Nom del fitxer d'avatar seleccionat

2.2 Relacions entre entitats

El diagrama lògic de relacions és el següent:

- **Game** ↔ **Shop**: relació Many-to-Many a través d'Availability (amb atributs de preu i URL de l'oferta).
- **Game** ↔ **Genre**: relació Many-to-Many directa (un joc pot tenir diversos gèneres).
- **Game** ↔ **Platform**: relació Many-to-Many directa (un joc pot estar a diverses plataformes).
- **User** ↔ **Game**: relació Many-to-Many a través de Wishlist (amb preu objectiu).
- **User** ↔ **Game**: relació Many-to-Many a través de Review (amb valoració i text).
- **User** ↔ **Profile**: relació One-to-One (cada usuari té un perfil únicament).

2.3 Decisions de disseny del model

Ús de dues APIs complementàries: CheapShark ofereix dades de preus en temps real però un catàleg limitat principalment a Steam. RAWG complementa el model amb més informació.

Degut a no poder accedir a jocs d'altres plataformes (ps5, Nintendo, Xbox…) desde les API's, vam decidir que la taula Platforms fes referència a la pàgina web que ven cada Joc (Steam, GOG…).

La funcionalitat de monitorització de preus desitjats per l'usuari de jocs seleccionats encara no ha estat implementada, però està previst en futures entregues, aixi com el preu mínim històric dels jocs.

Taules intermèdies amb atributs propis: tant Availability com Wishlist i Review no són simples taules de relació sinó que contenen atributs rellevants (preus, valoracions, dates). Cosa que justifica el seu tractament com a entitats independents en el model Django.

Ús del model User integrat de Django: per a la Wishlist i les Reviews s'ha optat per utilitzar el model d'usuari estàndard de Django en comptes de crear un model personalitzat. El model Profile s'associa via OneToOneField per afegir dades addicionals sense modificar el model base.

Constraint d'unicitat: Availability té una restricció unique_together sobre (game, shop) per evitar duplicats. Wishlist té la mateixa restricció sobre (user, game).

3. Arquitectura i estructura del projecte

El projecte segueix l'estructura estàndard de Django, organitzat en dos mòduls principals:

- **core/**: mòdul de configuració principal del projecte. Conté settings.py, urls.py, wsgi.py i asgi.py. La configuració sensible (SECRET_KEY, DEBUG, RAWG_API_KEY) es carrega des d'un fitxer .env mitjançant python-dotenv.
- **games/**: aplicació Django principal que conté els models, les vistes, les URLs, els templates, els fitxers estàtics i els scripts de càrrega de dades.

La càrrega de dades s'orquestra mitjançant dos mecanismes:

- **management/commands/load_deals.py**: comanda Django (python manage.py load_deals) que descarrega jocs i ofertes de CheapShark de manera paginada (8 pàgines × 60 jocs).
- **services.py**: mòdul de serveis que encapsula les crides a les APIs de RAWG (multimèdia) i CheapShark (dades de preus alternatives), mantenint la lògica de negoci separada de les vistes.

4. Adherència a les directrius 12-Factor App

L'aplicació ha estat dissenyada tenint en compte la metodologia 12-Factor App.

#	Factor	Descripció i compliment
I	Codebase	✅ Tot el codi és gestionat en un únic repositori Git públic a GitHub. No hi ha bifurcacions de codi entre entorns.
II	Dependencies	✅ Totes les dependències estan declarades explícitament a requirements.txt i aïllades dins del contenidor Docker.
III	Config	✅ La configuració sensible (SECRET_KEY, DEBUG, RAWG_API_KEY) s'emmagatzema en un fitxer .env ignorat pel control de versions. El settings.py la llegeix amb os.getenv().
IV	Backing Services	✅ La base de dades i les APIs externes (CheapShark, RAWG) es tracten com a recursos adjunts configurables. Canviar la BD només requereix modificar la configuració.
V	Build, Release, Run	✅ Docker separa estrictament la fase de construcció de la imatge (build) de l'execució dels contenidors (run). docker-compose gestiona el cicle complet.
VI	Processes	✅ L'aplicació Django s'executa com un procés sense estat. Totes les dades persistents es desen a la base de dades.
VII	Port binding	✅ L'aplicació és completament autònoma i exporta HTTP vinculant-se al port 8000 a través de Gunicorn, sense dependre d'un servidor web extern.
VIII	Concurrency	✅ Tots els models persisteixen l'estat a la base de dades, sense cap variable en memòria. Múltiples instàncies Docker poden executar-se en paral·lel sense conflictes.
IX	Disposability	✅ La naturalesa sense estat de l'aplicació i l'ús de contenidors Docker garanteixen arrencades ràpides i aturades netes.

#	Factor	Descripció i compliment
X	Dev/Prod Parity	✅ El codi utilitza exclusivament l'ORM estàndard de Django, sense cap sintaxi específica de SQLite. La mateixa base de codi funciona de manera idèntica en qualsevol motor de base de dades suportat.
XI	Logs	✅ L'aplicació no gestiona fitxers de log. Django escriu els esdeveniments directament a stdout, que Docker recull i gestiona.
XII	Admin Processes	✅ Les tasques administratives (migracions, càrrega de dades) s'executen com a processos puntuals en el mateix entorn que l'aplicació, via manage.py.

5. Autenticació i gestió d'usuaris

L'aplicació implementa un sistema complet d'autenticació, registre i gestió de perfil d'usuaris:

- **Autenticació**: s'utilitza una vista personalitzada (CustomLoginView) que estén LoginView de Django.
- **Registre**: s'ha implementat una vista de registre personalitzada (SignUpView) que utilitza CustomUserCreationForm afegint el camp email. En completar el registre l'usuari queda automàticament autenticat i es crea el seu Profile associat.
- **Logout**: gestionat per la vista integrada de Django amb redirecció a la pàgina principal.
- **Protecció de vistes**: les funcionalitats que requereixen autenticació (wishlist, perfil, ressenyes, eliminació de compte) estan protegides amb LoginRequiredMixin redirigint els usuaris no autenticats a la pàgina de login.
- **Perfil d'usuari**: vista ProfileView que mostra la wishlist, les últimes ressenyes, l'estalvi total acumulat i el nivell “Hunter”.
- **Edició de compte**: vista EditAccountView que permet modificar el nom d'usuari, la contrasenya i l'avatar. El canvi de contrasenya manté la sessió activa gràcies a update_session_auth_hash.
- **Eliminació de compte**: vista DeleteAccountView amb confirmació explícita abans de l'eliminació permanent.
- **Redireccionaments**: LOGIN_REDIRECT_URL i LOGOUT_REDIRECT_URL estan configurats a settings.py per dirigir l'usuari a la pàgina principal.

6. Conteneització amb Docker

L'aplicació inclou tots els fitxers necessaris per executar-se com a orquestració de contenidors Docker:

- **Dockerfile**: defineix la imatge de l'aplicació i instal·la les dependències de Python. També configura Gunicorn com a servidor WSGI de producció.
- **docker-compose.yml**: orquestra el servei web i mapeja el port 8000. Mmunta els volums necessaris per a la base de dades SQLite i els arxius estàtics i executa automàticament les migracions i la recopilació d'estàtics en arrencar.

La seqüència d'inici del contenidor és: migrate → collectstatic → gunicorn. Això garanteix que la base de dades estigui sempre actualitzada i els arxius estàtics disponibles abans que l'aplicació comenci a acceptar peticions.

7. Interfície d'usuari i navegació

- **Catàleg de jocs (Home)**: vista principal amb tots els jocs del catàleg paginats de 30 en 30. Inclou filtres per gènere, plataforma i rang de preus. També inclou opcions d'ordenació per preu (ascendent/descendent), puntuació Metacritic i novetat. La barra lateral mostra el recompte de jocs per cada categoria i plataforma.
- **Detall de joc**: vista individual d'un joc amb la seva portada, puntuació en estrelles, descripció obtinguda de RAWG, gèneres, plataformes, captures de pantalla ampliables i tràiler de vídeo. Mostra el preu actual i l'enllaç directe a la botiga. Inclou el sistema de comentaris i valoracions.
- **Perfil d'usuari**: pàgina personalitzada per a usuaris autenticats amb avatar animat, nivell Hunter, estadístiques (jocs a la wishlist, ressenyes escrites, estalvi total), la wishlist completa i les últimes ressenyes fetes.
- **Edició de compte**: formulari per modificar nom d'usuari, contrasenya i avatar (6 opcions disponibles).
- **Eliminació de compte**: pàgina de confirmació abans de l'eliminació permanent del compte i totes les dades associades.
- **Login / Registre**: formularis d'accés i creació de compte amb validació de contrasenyes.

8. Instruccions de desplegament

8.1 Opció A — Execució local

1. Clonar el repositori: git clone https://github.com/DealHunterOrgan/DealHunter.git
2. Accedir al directori: cd DealHunter
3. Crear i activar l'entorn virtual: python -m venv dhunter && source dhunter/bin/activate
4. Instal·lar dependències: pip install -r requirements.txt
5. Crear el fitxer .env a l'arrel amb les variables: SECRET_KEY, RAWG_API_KEY
6. Aplicar migracions: python manage.py migrate
7. Carregar dades: python manage.py load_deals
8. Recopilar estàtics: python manage.py collectstatic
9. Arrencar el servidor: python manage.py runserver
10. Accedir a l'aplicació a: http://localhost:8000

8.2 Opció B — Desplegament amb Docker

11. Assegurar-se que Docker i Docker Compose estan instal·lats.

12. Crear el fitxer .env a l'arrel del projecte amb les variables requerides.
13. Al directori arrel del projecte, executar: docker-compose up --build
14. Accedir a l'aplicació a: http://localhost:8000
15. Per aturar els contenidors: docker-compose down

8.3 Opció C — Accés Públic

L'aplicació és accessible públicament mitjançant la següent URL:
https://dealhunter-e1zt.onrender.com/